

微前端的沙盒技术体系

在字节跳动的落地实践

微前端的沙盒技术体系

目录

①沙盒应该做什么

②沙盒像什么

③沙盒怎么做

④字节跳动的沙盒做了什么

⑤沙盒模式中埋点、系统采样的设计

- Code Splitting

- dynamic import
 - react-lazy (react-loadable)
- webpack 4.0
 - Named chunk
 - ...

- Runtime Splitting

- Iframe
- sandbox

- Deployment Splitting

① 沙盒应该做什么

古老的 iframe —— 古老的困难

一个站点页面拆成 N 个 frame

每个 frame 单独一个独立域名

独立上下线

独立运行时

难以 deeplinking

共用数据的困难：

登录身份、站内信、跨模块通信

困难重重的共用代码、加载优化、运行优化

② 沙盒像什么

1. Docker

- 开发者必须体会不到环境区别
- 运行时没有环境差异
- 服务端微服务的基石

② 沙盒像什么

2. Docker 时代之前的(服务端)微服务

- 虚拟机使用复杂，维护成本巨大
 - 资源消耗
 - 镜像启动
 - 进程通信
- 直到 Docker 普及
- 前端的“微服务”在浏览器环境下并没有

② 沙盒像什么

3. 微前端的情况

- 前端沙盒像浏览器 Docker
- iframe 像虚拟机
- 字节跳动使用前端沙盒

③ 沙盒怎么做

1.参考单核、操作系统进程模拟进程切换策略

- JavaScript 是单线程的
- 通过对路由切换的封装，模拟单进程
- 通过对事件循环封装，模拟单核多进程

③ 沙盒怎么做

2. 用 Context 切换模拟线程安全

新沙盒即将激活时，查找当前激活中的沙盒

保存现场，存储 context

恢复之前的 context

③ 沙盒怎么做

3. Context 切换的笛卡尔积

- 比较并切换
- 沙盒数量 N 的笛卡尔平方
- 退回“初始” context
- 恢复之前 diff 的 context

④ 字节跳动的沙盒做什么

1. CSS 的干扰

①独立开发、混合加载

HTML 标准的 CSS 作用域

Scoped CSS

Shadow DOM

②CSS module CSS in JS

DOM header

③单核多进程的情况

④CSSStyleSheet.cssRules

④ 字节跳动的沙盒做什么

2. 全局变量的干扰

①Polyfill 等差异巨大

例如 generatorRuntime

组件模块化

全局的外部环境

②Identifier

let

const

class

③ Configurable

④window.location

④ 字节跳动的沙盒做什么

3. 所有需要进程安全的对象

DOM 沙盒等

Cookie

localStorage

⑤ 沙盒下的埋点、统计等

1. 埋点数据的缓存创建

- 全局数据（uid 等）默认缓存本地
- 缓存跟随沙盒切换
- 两级缓存：沙箱内全局、系统全局

⑤ 沙盒下的埋点、统计等

2.埋点数据的发送

- 异步发送
- 触发时机在沙盒外、缓存跟随沙盒切换
- 全局缓存和本地缓存统一本地存储

⑤字节跳动沙盒下的埋点、统计等

3.Console 回收

干净体面

控制 sourceMapping

向 log 中注入 callstack

额外加入快照

⑤字节跳动沙盒下的埋点、统计等

4. sourceMapping

原理：文本的映射

new Function

管理发布

字节跳动的微前端沙盒实践

<https://mp.weixin.qq.com/s/VlCK65cT7XikzFJpyks9Yg>



前端微服务在字节跳动的打磨与应用

<https://mp.weixin.qq.com/s/iLdAH9p2-S8pFyZrNzYaNg>



欢迎关注「字节跳动技术团队」

谢谢大家
aishiguang@bytedance.com

